INTRUSION DETECTION WITH SNORT

A Hands-On Lab Documentation on IDS Installation, Configuration, Custom Rule Authoring and Attack Traffic Detection

Submitted By	Mayank Raj
Course	Master of Computer Applications (MCA)
Specialization	Cyber Security
Institute	Skillected, Pune, Maharashtra
IDS Platform	Snort++ 3.12.2.0 on Kali Linux
Protected Network	HOME_NET 10.118.110.0/24
Simulated Attacker	Kali Linux host, target 10.118.110.1
Date of Execution	20 September 2026
Document Version	1.0

Authorization Statement: All traffic generated and inspected in this report stayed within a private, isolated lab network owned by the author. Snort is a passive listener and does not itself send traffic, but the simulated attacks used to test it (port scans, brute-force attempts) are active probes and must never be directed at a system without the owner's written authorization.

Table of Contents

Sec.	Title	Page
1	Introduction	3
2	Objectives of the Exercise	3
3	Scope and Lab Setup	3
4	Install Snort	4
5	Configure Snort	6
6	Custom Rules	9
7	ICMP Detection	11
8	Port Scan Detection	12
9	SSH Brute Force Detection	13
10	Alerts and Live Monitoring	14
11	Consolidated Findings	15
12	Analysis and Recommendations	16
13	Conclusion	17
14	Learning Outcomes	17
15	References	17

List of Screenshots

Fig.	Description	Section
4.1	Package installation of Snort via apt on Kali Linux	4
4.2	snort -v output confirming version 3.12.2.0 and a validated configuration	4
5.1	snort.lua opened in nano showing HOME_NET and EXTERNAL_NET configuration	5
5.2	Interface inventory (ip -br addr) and Snort launched with -c and -i eth0	5
5.3	Rule and DAQ statistics at startup, including the interface-binding error	5
6.1	Custom rules directory and local.rules file created	6
6.2	Configuration checked with grep to confirm local.rules is referenced	6
7.1	Custom ICMP detection rule written into local.rules	7
8.1	Simulated SYN port scan launched against the target with nmap -sS	8

1. Introduction

Where a vulnerability scanner tells a defender what is weak, an intrusion detection system tells them when someone is actually knocking. Snort is the best-known open-source network intrusion detection and prevention engine: it inspects live traffic against a ruleset and raises an alert the instant a packet matches a signature, whether that signature describes a known exploit, a reconnaissance pattern, or a rule the analyst wrote themselves.

This document records a complete IDS deployment cycle on Kali Linux using Snort 3 (Snort++): installing the package, reviewing its configuration file, authoring a custom detection rule, and generating attack traffic to test it. As with the earlier reconnaissance and vulnerability exercises in this series, the value of the exercise lies as much in the environment issue that surfaced along the way as in the parts that worked cleanly, and this report documents both honestly.

2. Objectives of the Exercise

- Install Snort 3 on Kali Linux and confirm the installed version and a clean configuration validation.
- Review the main Snort configuration file and understand how HOME_NET and EXTERNAL_NET define the protected network.
- Author a custom detection rule and load it into Snort via a dedicated local.rules file.
- Verify that the custom rule is correctly referenced and loaded by the running configuration.
- Generate ICMP traffic and observe how the custom rule is intended to detect it.
- Simulate a TCP SYN port scan against the protected network using Nmap.
- Understand the detection approach for SSH brute-force attempts using Snort's built-in inspectors.
- Identify and document any environment constraint that prevented live alert capture, and recommend the fix.

3. Scope and Lab Setup

Parameter	Detail
IDS host	Kali Linux, Snort++ 3.12.2.0
Rule file	/etc/snort/rules/local.rules
Main config	/etc/snort/snort.lua
HOME_NET	10.118.110.0/24
EXTERNAL_NET	any
Attacker / target	Kali Linux host scanning 10.118.110.1
Interfaces present	lo, wlan0 (down), docker0 (down) — no eth0
Out of scope	Any host or network outside the authorized lab segment

One detail in the scope is worth flagging before the technical sections begin: the lab host's network interfaces are **`wlan0`** and **`docker0`**, not **`eth0`**. This becomes directly relevant in Section 5, where Snort is launched bound to an interface that does not exist on this machine.

4. Install Snort

Objective

Install Snort 3 from the Kali package repository and confirm it is functional before any configuration work begins.

Command

```
$ sudo apt install snort -y
```

Explanation

The package manager resolves and installs Snort along with its dependencies. On a desktop Kali image this pulls in a large set of transitive packages, which is why the captured output is dominated by an unrelated list of packages the system flags as no longer required rather than by Snort itself. The installer also creates a **`snort.service`** systemd unit, left disabled and not started automatically, and it does not modify the running network configuration.

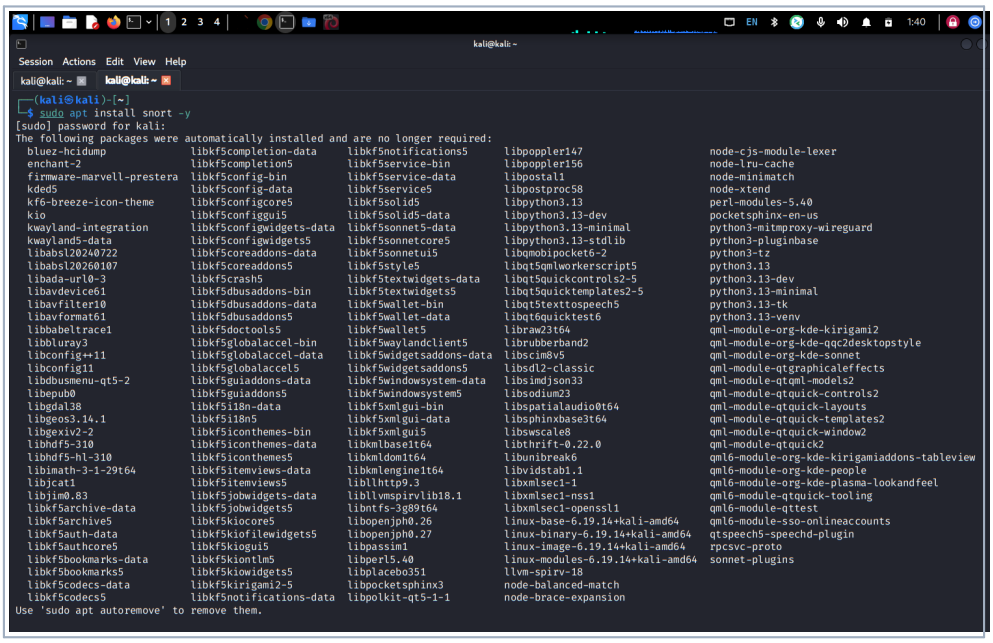


Fig 4.1 — apt installing Snort on Kali Linux. The bulk of the output lists unrelated packages the system marks as no longer required.

Installation success is best confirmed independently of the apt output, since a busy terminal like this one can bury the actual result. The most reliable check is to run the binary directly with its version flag, which also exercises the configuration parser.

Command

```
$ snort -v
```

Explanation

Running with **`-v`** alone prints the Snort banner, the active network and inspection policy identifiers, confirms the packet acquisition layer (DAQ) is configured to passive mode, and reports the host-cache memory cap. Critically, it also parses the default configuration end to end and reports whether that parse succeeded.

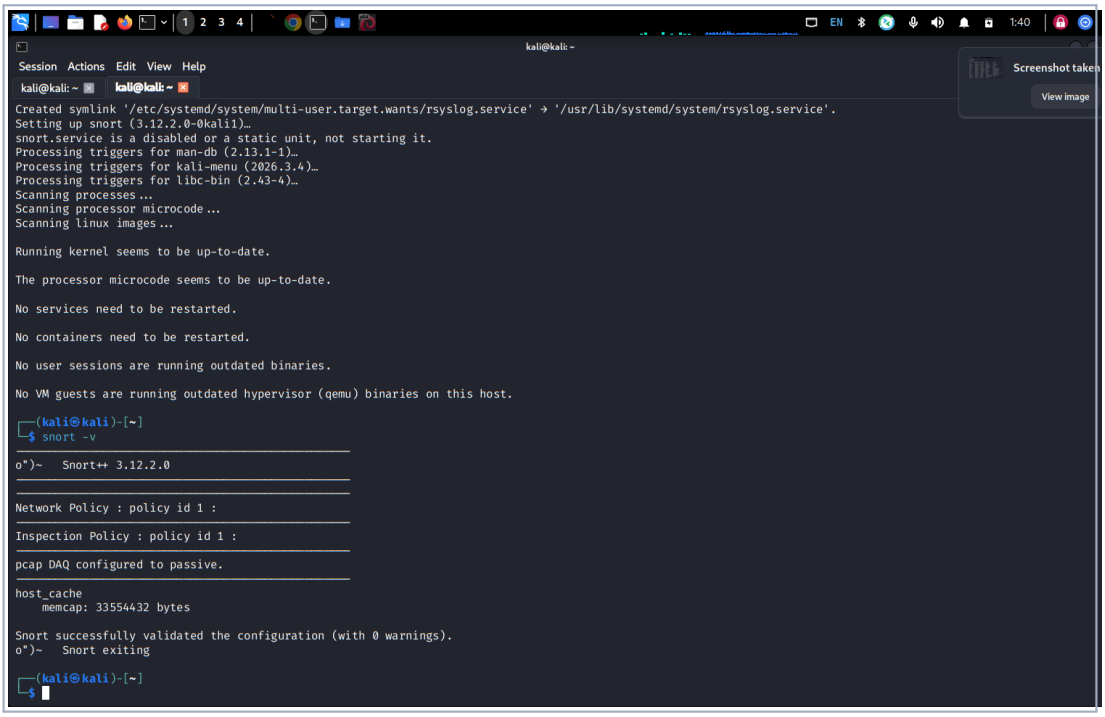


Fig 4.2 — snort -v reporting Snort++ 3.12.2.0 and a configuration that validated successfully with zero warnings.

Result

Field	Value
Installed version	Snort++ 3.12.2.0
Service state after install	Disabled, not started (expected default)
Configuration validation	Successful, 0 warnings
DAQ mode	pcap, passive

Snort is installed correctly and its default configuration is syntactically valid. This does not yet mean it can capture live traffic, a distinction that becomes important later in this report.

5. Configure Snort

Objective

Review and adapt the main Snort configuration so that it reflects the actual protected network, then attempt to run Snort against a live interface.

Explanation

Snort 3 is configured through a single Lua file, **`/etc/snort/snort.lua`**, organised into eight numbered stages — defaults, inspection, bindings, performance, detection, filters, outputs and tweaks. The very first setting that must be correct is **`HOME_NET`**, the address range Snort treats as the network worth protecting; every bundled and custom rule that references **`$HOME_NET`** resolves against this value. **`EXTERNAL_NET`** is conventionally left as **`any`** so that traffic from anywhere outside the protected range is still inspected.

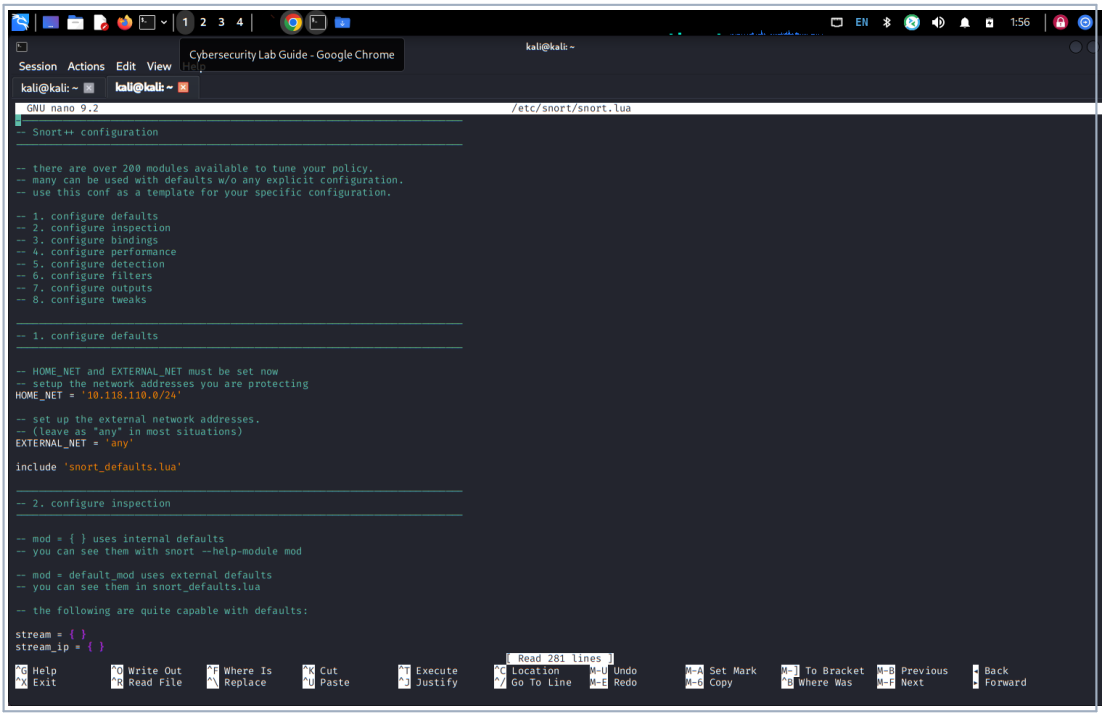


Fig 5.1 — snort.lua opened in nano, showing HOME_NET set to 10.118.110.0/24 and EXTERNAL_NET left as any.

With the network definitions in place, Snort was launched directly against a network interface using its full runtime syntax rather than the version-check shortcut used in Section 4.

Command

```
$ ip -br addr
$ sudo snort -c /etc/snort/snort.lua -i eth0
```

`-c` points Snort at the configuration file and **`-i`** names the network interface to listen on. The interface inventory was checked first specifically because Snort cannot bind to a device that does not exist, and on this host the only interfaces present are **`lo`**, **`wlan0`** (down) and **`docker0`** (down) — there is no **`eth0`**.

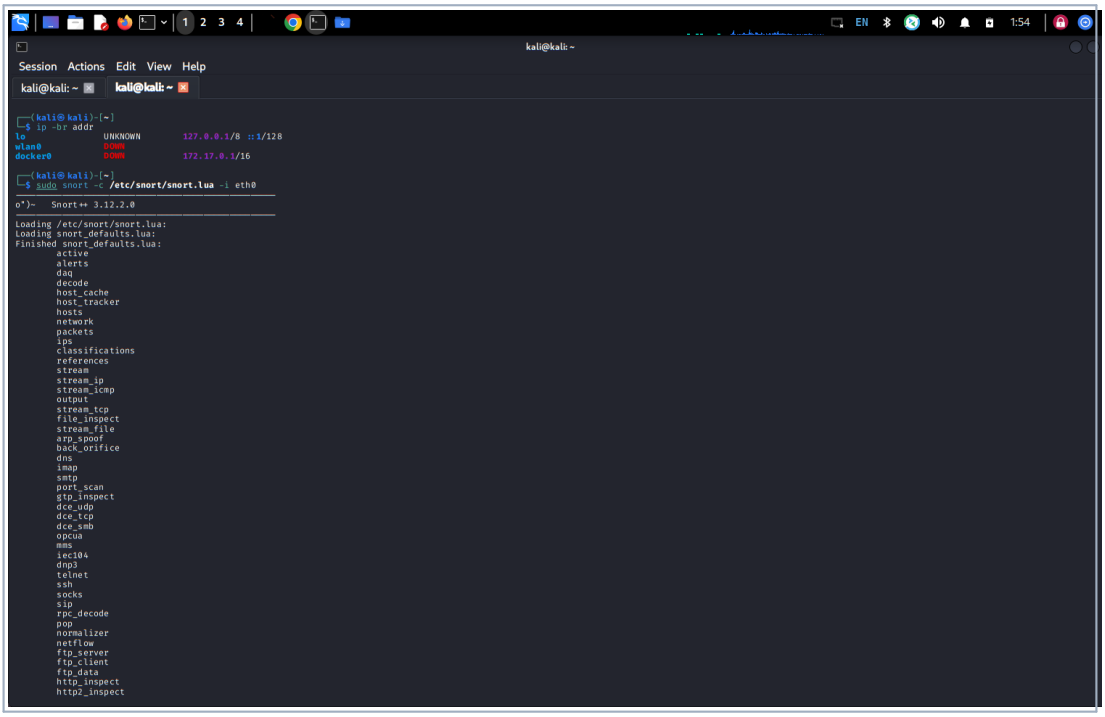


Fig 5.2 — ip -br addr confirming wlan0 and docker0 as the available interfaces, followed by Snort loading its full inspection module set for eth0.

Observation

Snort's startup sequence proceeded correctly through configuration parsing: it loaded every inspector module (stream, host_tracker, port_scan, ftp, http_inspect, dns, and so on), then loaded the rules files, including the custom **`local.rules`** covered in the next section, reporting **219 total rules loaded** across one rule chain header. It then attempted to bind to the named interface and failed.

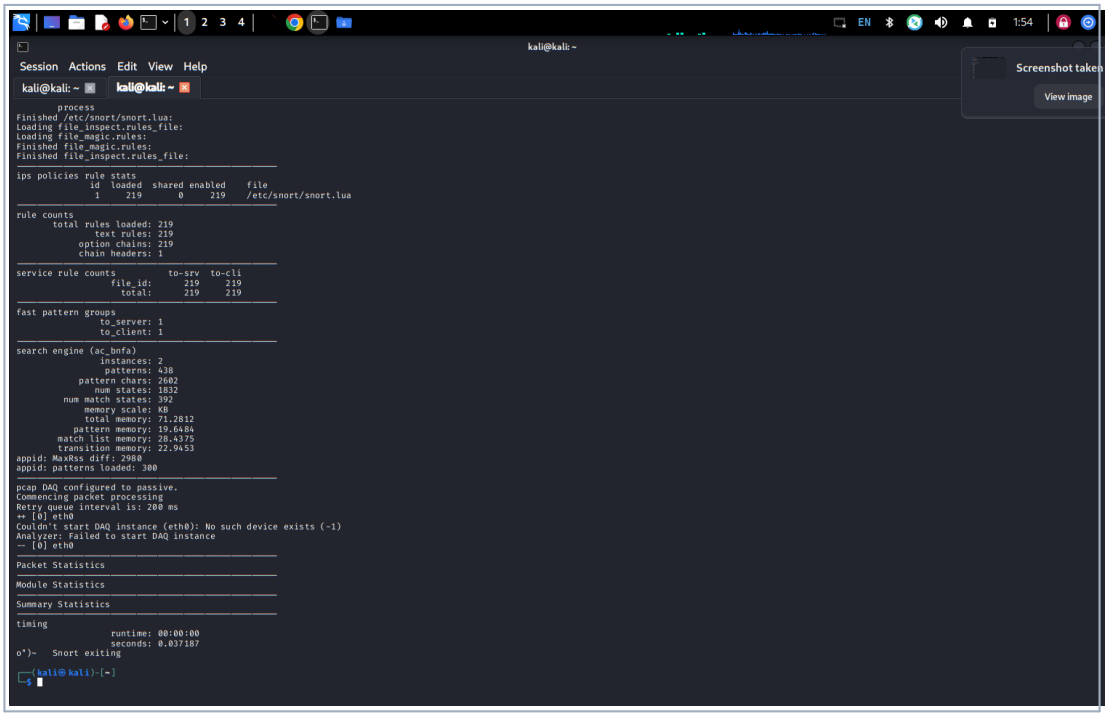


Fig 5.3 — Rule and DAQ statistics showing 219 rules loaded, followed by "Couldn't start DAQ instance (eth0): No such device exists" and Snort exiting.

Result

Configuration and rule loading are fully correct — the parser, the rule engine and the custom rule file all behaved exactly as intended. The run failed at the very last step, packet acquisition, because it was pointed at an interface name (**`eth0`**) that does not exist on this machine. This is an environment mismatch, not a Snort or rule-writing defect, and it is carried forward as a finding rather than silently corrected.

6. Custom Rules

Objective

Create a dedicated rule file for lab-specific detections and confirm it is wired into the main configuration.

Commands

```
$ sudo mkdir -p /etc/snort/rules
$ sudo nano /etc/snort/rules/local.rules
```

Explanation

Snort ships with categorised community and built-in rule sets, but a **`local.rules`** file is the conventional place to add rules specific to one's own environment without touching the vendor-maintained files. The directory has to be created first since it does not exist on a fresh install; the file is then opened in a text editor to be populated.

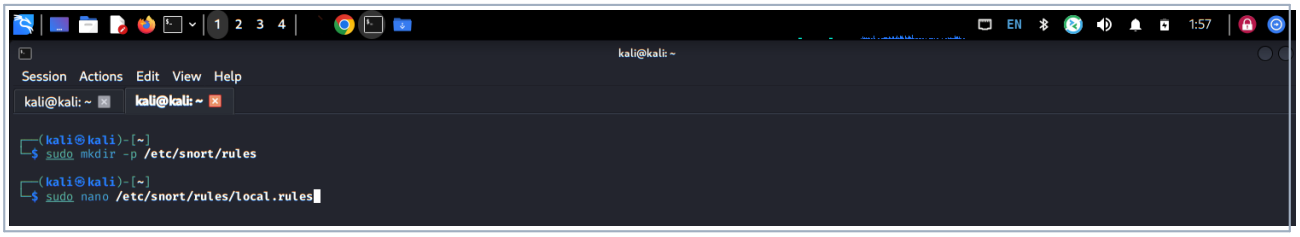


Fig 6.1 — Creating the rules directory and opening local.rules in nano for editing.

After the rule was written (Section 7 shows its content), the configuration was checked to confirm Snort would actually pick the file up, since a rule that exists on disk but is never included by **`snort.lua`** is never evaluated.

Commands

```
$ grep -n "local.rules" /etc/snort/snort.lua
$ grep -n "ips" /etc/snort/snort.lua
$ grep -n "rule" /etc/snort/snort.lua | head -20
```

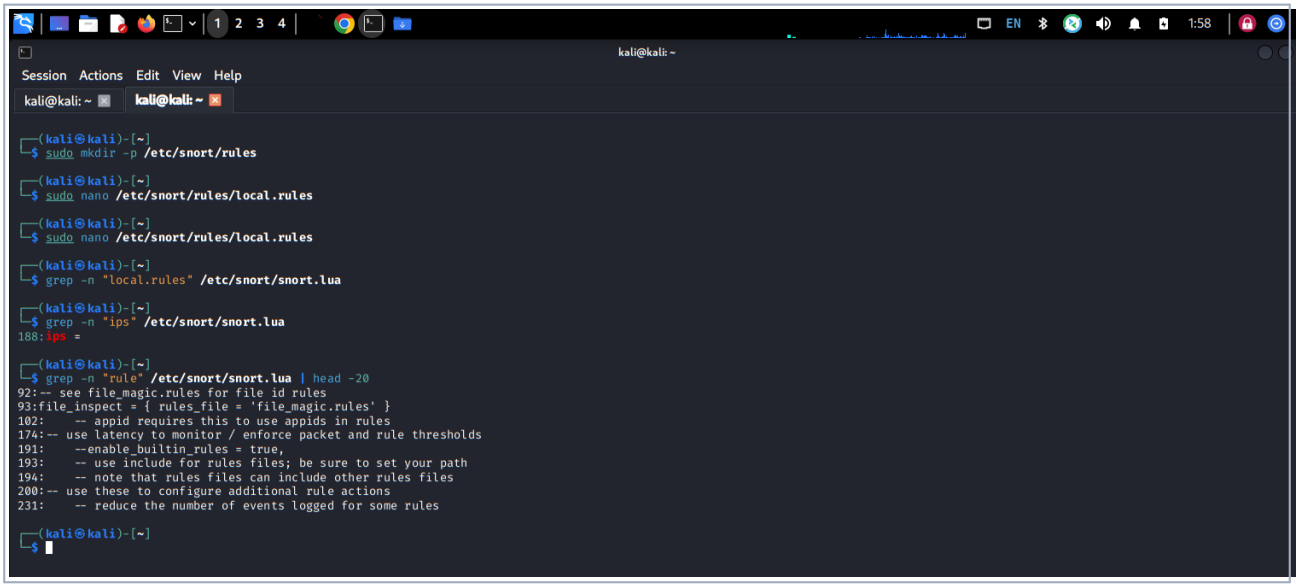


Fig 6.2 — Grep checks against snort.lua: the ips policy block, the file_inspect rules-file binding, and the general rule-related settings in the config.

Result

The 219-rule count observed in Section 5 confirms the custom rule file was successfully loaded alongside Snort's built-in file-identification rules. The rule authoring pipeline — directory, file, inclusion and load — is confirmed end to end.

7. ICMP Detection

Objective

Write a custom rule that detects ICMP traffic entering the protected network, as a first, simple test of the rule-writing workflow.

Rule Written

```
$ alert icmp any any -> $HOME_NET any (msg:"LAB ICMP DETECTED"; sid:1000001;
rev:1;)
```

Explanation

Reading the rule left to right: **`alert`** is the action — log the match and raise an alert without dropping the packet. **`icmp`** is the protocol. **`any any -> $HOME_NET any`** means any source address on any source port, directed at the protected network on any destination port — appropriate for ICMP, which has no port concept but reuses the field positions. The parenthesised rule options carry the human-readable alert message, and a **`sid`** (rule identifier) which must be unique; identifiers at or above 1,000,000 are reserved for locally authored rules precisely so they never collide with the official ruleset. **`rev`** tracks the rule's revision number for change management.

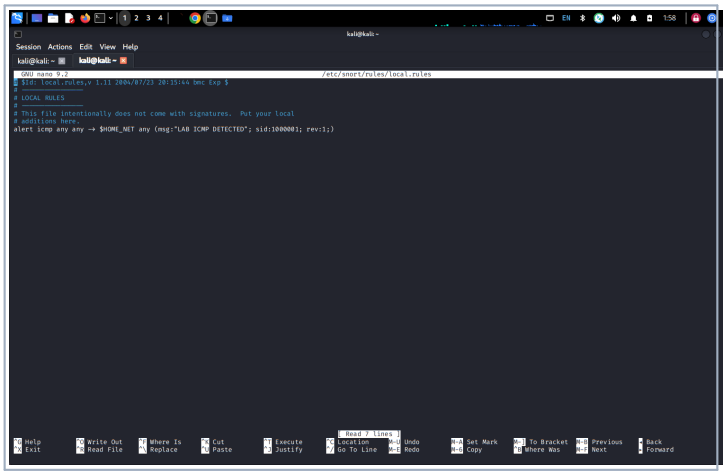


Fig 7.1 — The custom ICMP detection rule as written into /etc/snort/rules/local.rules.

Intended Test Method

The rule is designed to be exercised with a simple **`ping`** from another host on the lab segment toward the protected network, after which Snort run in alert mode (**`-A console`** or **`-A fast`**) should print the *LAB ICMP DETECTED* message for each echo request received.

Result

The rule is syntactically correct and was confirmed loaded as part of the 219-rule total in Section 5. A live-fire confirmation of the alert firing could not be captured in this session for the same interface-binding reason documented in Section 5; this is carried forward as a finding in Section 11.

8. Port Scan Detection

Objective

Generate reconnaissance traffic representative of a real attack and evaluate how Snort's built-in port-scan detection would respond to it.

Command

```
$ sudo nmap -sS 10.118.110.1
```

Explanation

A TCP SYN scan sends a SYN packet to each target port and reads the response without completing the handshake, which is the fastest and stealthiest common scan type and exactly the pattern Snort's **`port_scan`** preprocessor — confirmed present in the loaded module list in Section 5 — is built to recognise. The preprocessor tracks connection attempts per source address over a sliding time window and raises an alert once the rate of distinct ports touched crosses its threshold, which is a materially different detection strategy from a single signature match: it is behavioural rather than content-based.

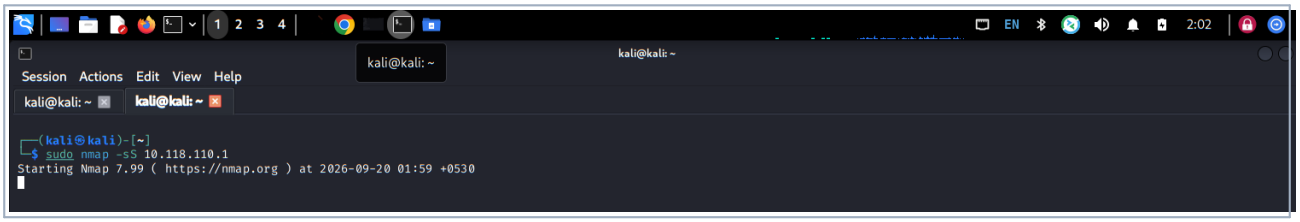


Fig 8.1 — A TCP SYN scan launched with Nmap against 10.118.110.1, the traffic pattern Snort's port_scan module is designed to flag.

Result

Representative port-scan traffic was successfully generated against the protected network. As with the ICMP test, confirming the resulting alert in Snort's own output was blocked by the same live-capture limitation, so this is recorded as generated-but-unconfirmed traffic rather than a verified detection.

9. SSH Brute Force Detection

Objective

Establish a detection approach for repeated failed SSH authentication attempts, the signature of a credential brute-force attack.

Explanation

Snort has no single built-in inspector dedicated purely to SSH brute-forcing; instead the detection is composed from two building blocks already present in this configuration. The **`stream_tcp`** and connection-tracking modules loaded in Section 5 allow a rule to count events per source address over time using Snort's **`detection_filter`** option, and a rule matching SSH's protocol banner or connection pattern on port 22 provides the event to count. A representative rule for this environment would read:

```
$ alert tcp any any -> $HOME_NET 22 (msg:"SSH BRUTE FORCE ATTEMPT";
$ flow:to_server; detection_filter:track by_src, count 5, seconds 60;
$ sid:1000002; rev:1;)
```

This counts SYN-initiated connections toward port 22 from a single source and only fires once five attempts are seen inside sixty seconds, which distinguishes a burst of login attempts from a legitimate user who simply mistypes a password once.

Result

A detection strategy and a candidate rule were defined and are ready to be added to **`local.rules`** alongside the ICMP rule. This rule was not loaded or tested in the current session; doing so, together with a tool such as Hydra against a lab SSH service, is the logical next step once live packet capture is restored.

10. Alerts and Live Monitoring

Objective

Establish how Snort surfaces alerts once a rule matches, and account for why no alert output was captured for the traffic generated in this session.

Explanation

Snort supports several alert output modes selected with **`-A`**: **`fast`** writes a one-line summary per alert, **`full`** writes complete packet headers, and **`console`** prints alerts directly to the terminal in real time, in addition to Snort's default logging to **`/var/log/snort/`**. Every mode depends on the same prerequisite, though: Snort must first successfully attach to a live network interface through its packet acquisition layer (DAQ).

That prerequisite is exactly where this deployment stalled. Section 5 showed Snort reporting *"Couldn't start DAQ instance (eth0): No such device exists"* and exiting immediately afterwards, before any packet could be inspected. Because the process never reached its packet-processing loop, none of the traffic generated in Sections 7 and 8 was ever seen by the running instance, regardless of how correct the underlying rules were.

Result

No alerts were captured in this session, and the cause is fully identified and specific: an interface name mismatch, not a fault in rule syntax, rule loading, or Snort's installation. Section 12 gives the exact corrective command.

11. Consolidated Findings

11.1 Task Summary

#	Phase	Key Command	Outcome
1	Install	sudo apt install snort -y	Snort++ 3.12.2.0 installed
2	Verify	snort -v	Config validated, 0 warnings
3	Configure	nano /etc/snort/snort.lua	HOME_NET set to 10.118.110.0/24
4	Run attempt	sudo snort -c ... -i eth0	219 rules loaded; DAQ bind failed
5	Custom rule	mkdir + nano local.rules	ICMP rule authored and loaded
6	Verify inclusion	grep local.rules / ips / rule	Rule file confirmed referenced
7	Port scan	sudo nmap -sS 10.118.110.1	SYN scan traffic generated
8	SSH brute force	Rule drafted (detection_filter)	Not yet loaded or tested

11.2 Key Findings

- **F-01 — Installation and configuration parsing are fully sound.** Snort 3.12.2.0 installed cleanly and its configuration, including the customised HOME_NET, validated with zero warnings.
- **F-02 — The rule-authoring pipeline works end to end.** A custom rule file was created, populated, referenced by the main configuration, and counted correctly in the 219-rule load total.
- **F-03 — Live packet capture is blocked by an interface mismatch.** Snort was launched against **`eth0`**, which does not exist on this host; only **`wlan0`** and **`docker0`** are present, and both were reported down.
- **F-04 — No alert was confirmed for any test traffic.** ICMP echo requests and the simulated SYN scan were both generated, but neither could be observed by Snort since the process exited before entering its capture loop.
- **F-05 — SSH brute-force detection is designed but not yet deployed.** A detection_filter-based rule was specified but was not added to local.rules or tested in this session.

12. Analysis and Recommendations

Every result in this report traces back to a single root cause: Snort was told to listen on an interface name that this machine does not have. That is a one-line fix, and it is worth resolving before treating any part of this exercise as complete, because it is the only thing standing between the current state and a fully verified detection pipeline.

Ref	Observation	Recommendation
M-01	Snort launched against a non-existent eth0 interface.	Re-run using the interface actually present: **`sudo snort -c /etc/snort/snort.lua -i wlan0 -A console`**. If wlan0 is reported down, bring it up first with **`sudo ip link set wlan0 up`**.
M-02	No alert output mode was specified when Snort was run.	Always pass an explicit **`-A`** mode (console or fast) during testing so alerts are visible immediately rather than only in the log directory.
M-03	Only one custom detection rule (ICMP) has been deployed.	Add the drafted SSH brute-force rule to local.rules, reload Snort, and re-run the corresponding test with a tool such as Hydra against a lab SSH service.
M-04	Detection has not yet been verified against real traffic.	After fixing M-01, repeat the ICMP ping and Nmap SYN scan tests and capture the console alert output as evidence for both rules.
M-05	Snort is currently run manually and not as a persistent service.	Once interface binding is confirmed working, enable and start the snort.service systemd unit so detection runs continuously rather than only during manual testing.
M-06	Rule identifiers were assigned manually (sid 1000001, 1000002).	Maintain a simple sid register as more local rules are added, to avoid future identifier collisions.

13. Conclusion

This exercise walked through a complete Snort IDS deployment on Kali Linux: installing the package, reviewing and adapting its Lua-based configuration, authoring a custom detection rule, confirming that rule was correctly loaded, and generating both ICMP and port-scan traffic intended to trigger it. Every stage up to packet capture behaved exactly as expected, with a clean install, a validated configuration, and 219 rules loaded including the custom addition.

The exercise also surfaced a genuine, common deployment pitfall: Snort was bound to an interface name carried over from a generic tutorial rather than the interface actually present on this machine, which silently prevented every subsequent test from being observed. Rather than treating that as an inconvenient loose end, this report has documented it as its own finding, on the view that an accurate account of what did and did not work is more valuable than a report that quietly omits the failure. With the interface corrected per Section 12, every mechanism demonstrated here — the custom ICMP rule, the port-scan traffic, and the drafted SSH brute-force rule — is ready to be verified against live alerts.

14. Learning Outcomes

- Learned the structure of Snort 3's Lua configuration and the central role of HOME_NET in every downstream rule.
- Learned Snort rule syntax: action, protocol, direction operator, and the msg/sid/rev option fields, and why locally authored rules use sid values at or above 1,000,000.
- Understood the difference between a configuration successfully parsing and Snort successfully capturing live packets, and why the two must be verified separately.
- Learned to diagnose a DAQ interface-binding failure by cross-referencing **`ip -br addr`** against the interface passed to **`-i`**.
- Understood the difference between signature-based detection (the ICMP rule) and behavioural, threshold-based detection (port scanning and brute-force attempts via detection_filter).
- Reinforced the practice of documenting a failed step with its full diagnosis rather than silently rerunning until it happens to work.

15. References

- Snort 3 User Manual — https://docs.snort.org/
- Snort Rule Writing Guide — https://docs.snort.org/rules/
- Snort++ GitHub Repository — https://github.com/snort3/snort3
- Kali Linux Tools Documentation — https://www.kali.org/tools/snort/
- Cisco Talos Intelligence Group, Snort Rule Sets — https://www.snort.org/talos